

**INGENIERÍA Y CALIDAD
DE SOFTWARE**

REQUERIMIENTOS ÁGILES

User Stories · Criterios de aceptación · INVEST
Spikes · Story Points · Planning Poker · Velocity

APUNTE UNIVERSITARIO

UTN — Facultad Regional Córdoba
Ciclo Lectivo 2026 · Segundo Cuatrimestre

Basado en el material de clase proporcionado

Índice

1. Filosofía Ágil	3
2. Requerimientos en Agile	4
3. Tipos y niveles de requerimientos	5
4. User Stories	6
5. Criterios y pruebas de aceptación	8
6. INVEST y calidad de las historias	10
7. Spikes	11
8. Estimaciones en ambientes ágiles	12
9. Story Points y escalas	13
10. Planning Poker	14
11. Velocity y compromisos	15
12. Integración de conceptos	16
13. Repaso para parcial	17
Fuentes del material	18

Cómo usar este apunte

Primero entendé la idea central de cada sección; después repasá los ejemplos y, al final, usá el apartado “Repaso para parcial” para comprobar si podés explicar los conceptos sin mirar la respuesta.

1. Filosofía Ágil

Ágil no es una metodología o proceso único. Es una ideología con principios que guían el desarrollo del producto y que priorizan la adaptación, la colaboración, las entregas frecuentes y el valor.

La propuesta busca un equilibrio entre ningún proceso y demasiado proceso. Por eso, el enfoque ágil se caracteriza por ser adaptable en lugar de predictivo y orientado a las personas en lugar de orientado al proceso.

- Planificación continua y multinivel.
- Equipos facultados, autoorganizados y completos.
- Entregas frecuentes, iterativas y priorizadas.
- Prácticas de ingeniería disciplinadas.
- Integración continua y testing concurrente.

Ser ágil no significa ser indisciplinado

Dividir el trabajo en partes pequeñas no alcanza. La agilidad implica disciplina, aprendizaje, feedback, adaptación y foco permanente en el valor.

1.1. La idea central: valor y adaptación

En Agile, lo importante no es simplemente entregar una salida o un requerimiento. El objetivo es entregar un resultado, es decir, una solución que produzca valor.

“Los cambios son la única constante.”

Los cambios en los requerimientos pueden transformarse en una ventaja competitiva cuando el equipo tiene la capacidad de actuar sobre ellos. Esto lleva a priorizar y repriorizar el trabajo de manera continua.

2. Requerimientos en Agile

La gestión ágil de requerimientos busca construir el producto correcto utilizando el valor como guía. Para comunicar qué construir se utilizan historias y modelos, y se intenta mantener el nivel de detalle en “sólo lo suficiente”.

- Usar el valor para construir el producto correcto.
- Usar historias y modelos para mostrar qué construir.
- Determinar “sólo lo suficiente” y profundizar el análisis cuando sea necesario.

2.1. BRUF y el costo del análisis anticipado

BRUF (Big Requirements Up Front) representa un enfoque en el que gran parte de los requerimientos se intenta definir por adelantado. El material contrasta esta lógica con Agile, donde se evita producir grandes cantidades de detalle que podrían quedar obsoletas por cambios posteriores.

Principio práctico

Analizar cuando se necesite, no antes. El análisis detallado se difiere hasta justo antes de que el trabajo

comience.

2.2. Gestión dinámica del backlog

- Las historias de mayor prioridad se implementan primero.
- Un requerimiento puede ser repriorizado.
- Un requerimiento puede ser removido si deja de aportar valor.
- Un nuevo requerimiento se incorpora y luego se prioriza.

3. Tipos y niveles de requerimientos

El material distingue niveles que van desde el problema y la necesidad de negocio hasta la solución concreta. Esta distinción ayuda a evitar confundir el objetivo con una decisión de implementación.

Tipo	Qué expresa	Ejemplo del material
Negocio	Objetivo o necesidad del negocio.	Disminuir el tiempo invertido en procesos manuales de atención.
Usuario	Necesidad que tiene el usuario.	Consultar en línea el estado de cuenta.
Funcional	Función que debe brindar el sistema.	Generar reportes de saldos o enviar notificaciones por mail.
No funcional	Restricción o atributo de calidad.	Formato PDF o requisitos de seguridad para credenciales.
Implementación	Decisión/condición concreta de la solución.	Servidores en la nube.

4. User Stories

Una User Story (Historia de Usuario) captura una necesidad desde la perspectiva del usuario. El material enfatiza que la tarjeta escrita no contiene toda la información: la conversación que se genera alrededor de ella es fundamental.

4.1. Las tres partes

- Tarjeta: captura la idea central.
- Conversación: permite descubrir y acordar detalles.
- Confirmación: permite comprobar que lo construido satisface lo acordado.

4.2. Formato

Plantilla

Como <nombre del rol>, yo puedo <actividad> de forma tal que <valor de negocio que recibo>.

El rol identifica quién realiza la acción o quién recibe el valor. La actividad representa lo que hará el sistema y el valor explica por qué esa actividad es necesaria.

4.3. Ejemplo

User Story

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

4.4. ¿Para qué sirve una User Story?

- Necesidad del usuario.
- Descripción del producto.
- Ítem de planificación.
- Token para una conversación.
- Mecanismo para diferir una conversación.

4.5. Historias como porciones verticales

Las User Stories se plantean como porciones verticales de funcionalidad: una historia puede impactar sobre GUI, lógica de negocio y base de datos, buscando representar un incremento completo de valor y no una tarea aislada de una sola capa.

4.6. Product Backlog y Product Owner

El Product Owner prioriza las historias en el Product Backlog. Cada iteración toma los ítems de mayor prioridad, mientras que las historias pueden ser reordenadas o eliminadas a medida que cambia el conocimiento del producto.

5. Criterios y pruebas de aceptación

5.1. Criterios de aceptación

Los criterios de aceptación definen los límites de una User Story. Permiten expresar los requerimientos funcionales mínimos necesarios para que la historia provea valor y ayudan a construir una visión compartida entre Product Owner, desarrollo y testing.

- Definen límites para una User Story.
- Ayudan al PO a expresar lo necesario para obtener valor.
- Ayudan al equipo a compartir una misma interpretación.
- Permiten derivar pruebas.
- Indican cuándo dejar de agregar funcionalidad a la historia.

Buenas prácticas

Un buen criterio define una intención, es independiente de la implementación y se mantiene relativamente en un nivel alto.

5.2. Ejemplo

Para la historia de búsqueda de destino, el material utiliza criterios como:

- La altura de la calle es un número.
- La búsqueda no puede demorar más de 30 segundos.

5.3. ¿Dónde van los detalles?

Detalles como el nombre exacto de una columna, un formato numérico específico o una decisión de interfaz pueden quedar en la documentación interna del equipo o en pruebas de aceptación automatizadas. La User Story no necesita convertirse en una especificación exhaustiva.

5.4. Pruebas de aceptación

Las pruebas de aceptación expresan detalles derivados de la conversación y complementan la historia. El material propone identificarlas primero y luego diseñar las pruebas completas.

Escenario	Resultado
País, ciudad, calle y altura existentes	Pasa
Calle inexistente	Falla
Altura inexistente	Falla
País inexistente	Falla
Ciudad inexistente	Falla
Búsqueda válida que tarda más de 30 s	Falla

6. INVEST y calidad de las historias

INVEST resume características útiles para evaluar si una User Story está bien planteada.

Letra	Concepto	Idea clave
I	Independent	Independiente y calendarizable cuando sea posible.
N	Negotiable	Se negocia el qué; no fija prematuramente el cómo.
V	Valuable	Debe aportar valor al cliente.
E	Estimatable	Debe poder estimarse para ayudar a ordenar el trabajo.
S	Small	Debe ser suficientemente pequeña para consumirse en una iteración.
T	Testable	Debe poder demostrarse mediante pruebas.

6.1. Historias que “huelen mal”

Una historia pierde el foco cuando describe directamente una tecnología, una herramienta, una arquitectura o una decisión concreta de implementación. Ejemplos del material incluyen “el software deberá estar desarrollado en JAVA” o “el sistema utilizará Log4J...”.

Regla práctica

Las User Stories se escriben desde la perspectiva del usuario y expresan intención. No hay que forzar todo a escribirse como User Story.

6.2. Un paso a la vez

El material recomienda evitar historias que intentan hacer demasiadas cosas al mismo tiempo. Una señal de alerta es el uso de múltiples acciones enlazadas con “Y”.

7. Spikes

Un Spike es un tipo especial de historia utilizado para reducir riesgo e incertidumbre sobre una User Story u otra faceta del proyecto. Puede ser técnico o funcional.

Tipo	Cuándo se usa	Ejemplos del material
Técnico	Existe incertidumbre sobre la tecnología o el dominio técnico.	Performance, decisión hacer/comprar, evaluar una tecnología, ancho de banda, push/pull.
Funcional	No está claro cómo debería interactuar el usuario con el sistema.	Prototipos y feedback sobre tamaño, estilo o presentación de una interfaz.

7.1. Lineamientos

- Deben ser estimables, demostrables y aceptables.
- Se reservan para incógnitas críticas y grandes.
- Son la excepción, no la regla.
- Como regla general, las spikes pueden ejecutarse en una iteración separada de las historias resultantes.
- Si el spike es pequeño, sencillo y la solución puede encontrarse rápidamente, puede incluirse en la misma iteración.

8. Estimaciones en ambientes ágiles

Las features y User Stories se estiman utilizando una medida de tamaño relativo conocida como ****Story Points (SP)****. El material remarca que una medida relativa no es absoluta y que Story Points no es una medida basada en tiempo.

8.1. ¿Por qué estimar de forma relativa?

- Las personas tienen dificultades para estimar en términos absolutos.
- Somos buenos comparando cosas.
- Comparar suele ser más rápido.
- Favorece la dinámica grupal y el pensamiento de equipo.
- Permite aprovechar mejor el tiempo de análisis de las historias.

8.2. Tamaño vs. esfuerzo

Tamaño se refiere a cuán grande o pequeña es una feature/story y a la cantidad de trabajo necesaria para producirla. El tamaño considera complejidad, trabajo requerido y magnitud de la historia.

Idea clave

Tamaño NO es esfuerzo. Las estimaciones en tiempo son sensibles a habilidades, conocimiento, supuestos, experiencia y familiaridad con el dominio.

8.3. ¿Qué es un Story Point?

Es una unidad de medida específica del equipo para representar el “peso” relativo de una historia considerando complejidad, riesgo y esfuerzo. No representa horas ni días.

9. Story Points y escalas

El equipo puede elegir diferentes escalas para representar tamaño relativo. Una vez elegida una escala, el material indica que no debe cambiarse, porque eso equivaldría a cambiar el “metro patrón”.

Escala	Ejemplo
Números	1 a 10
Talles	S, M, L, XL, XXL
Potencias de 2	1, 2, 4, 8, 16, 32, 64...
Fibonacci	0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...

9.1. ¿Cómo interpretar los valores?

SP	Interpretación aproximada
0	No hay suficiente conocimiento del producto o funcionalidad.
1/2 – 1	Funcionalidad pequeña, usualmente cosmética.
2 – 3	Funcionalidad pequeña a mediana.
5	Funcionalidad mediana.
8	Funcionalidad grande; conviene evaluar si puede dividirse.

SP	Interpretación aproximada
13	Hay que justificar por qué no puede dividirse.
20	Hay que justificar la razón de negocio y por qué no puede dividirse.
40	No habría forma razonable de realizarla en un sprint.
100	Señal de que algo está muy mal planteado.

Importante

Los Story Points son relativos y dependen del sistema de medición del equipo. No se deben convertir automáticamente en horas.

10. Planning Poker

Planning Poker es una técnica colaborativa de estimación popular en Agile y asociada a Mike Cohn. Combina opinión de experto, analogía y desagregación.

1. Definir la lista de features/stories a estimar y disponer de un mazo de cartas por estimador.
2. Elegir una Story base (canónica) para comparar las demás.
3. Leer la historia a todo el equipo.
4. Discutir dudas y consultar al Product Owner cuando sea necesario.
5. Cada estimador selecciona una carta y la coloca boca abajo.
6. Todos muestran las cartas al mismo tiempo.
7. Si hay diferencias, se discuten especialmente las estimaciones más altas y más bajas y se vuelve a votar.
8. La siguiente historia se estima por comparación con la historia base.

Pregunta que guía la comparación

¿Cuán grande, pequeña, compleja o riesgosa es esta Story comparada con la Story base?

11. Velocity y estimaciones

11.1. Velocity

Velocity es una métrica del progreso del equipo. Se calcula sumando los Story Points asignados a las User Stories que el equipo ****completa**** durante una iteración.

Regla fundamental

Se cuentan Story Points de historias completas, no de historias parcialmente completas.

La velocidad ayuda a observar cuánto trabajo termina realmente el equipo y sirve para corregir errores de estimación con el aprendizaje acumulado.

11.2. Estimaciones ≠ compromisos

El material advierte que utilizar estimaciones como compromisos puede ser peligroso y perjudicial para una organización. El mayor beneficio de estimar está en el proceso: conversar, detectar incertidumbre, descubrir riesgos y evaluar factibilidad.

“No ser dogmático sobre nada. Ser pragmático es la clave del éxito en casi todo.”

12. Integración de conceptos

Los conceptos de ambas clases se pueden entender como un flujo continuo de descubrimiento, decisión, construcción y aprendizaje:

Etapa	Qué sucede
Necesidad del usuario	Se identifica el problema o beneficio buscado.
User Story	Se expresa la necesidad desde la perspectiva del usuario.
Conversación	Se aclaran dudas y se descubre el detalle necesario.
Criterios de aceptación	Se delimitan las condiciones mínimas de satisfacción.
INVEST	Se evalúa la calidad y tamaño de la historia.
Spike (si hace falta)	Se reduce incertidumbre técnica o funcional.
Estimación	Se asignan Story Points por comparación relativa.
Priorización	El Product Owner ordena según valor.
Iteración	Se implementa y prueba el incremento.
Feedback	Se obtiene información para inspeccionar y adaptar.
Visión global Agile es empírico: inspeccionar y adaptar es mandatorio. El conocimiento obtenido durante el trabajo modifica decisiones posteriores.	

13. Repaso para parcial

Preguntas rápidas

Pregunta	Respuesta esperada
¿Qué es Agile?	Una ideología basada en principios, no una metodología única.
¿Qué problema busca evitar BRUF?	El costo de producir demasiado detalle de requerimientos demasiado temprano.
¿Qué es una User Story?	Una expresión de necesidad del usuario que sirve como tarjeta, conversación y punto de confirmación.

Pregunta	Respuesta esperada
¿Cuál es el formato básico?	Como [rol], quiero/puedo [actividad], para [valor].
¿Qué hacen los criterios de aceptación?	Definen límites de la historia y ayudan a derivar pruebas.
¿Qué significa INVEST?	Independent, Negotiable, Valuable, Estimatable, Small y Testable.
¿Qué es un Spike?	Una historia especial para reducir riesgo o incertidumbre.
¿Qué mide un Story Point?	Tamaño relativo, considerando complejidad, riesgo y esfuerzo.
¿Story Points son horas?	No.
¿Qué es Velocity?	La suma de SP de las historias completadas en una iteración.
¿Qué es Planning Poker?	Una técnica colaborativa de estimación por comparación y discusión.

Mapa mental de memoria

REQUERIMIENTOS ÁGILES

Valor → User Story → Conversación → Criterios de aceptación → Pruebas → Estimación relativa → Priorización → Iteración → Feedback → Inspección y adaptación

Errores típicos a evitar

- Confundir User Story con una especificación técnica exhaustiva.
- Escribir el cómo en lugar del qué/valor.
- Convertir Story Points directamente en horas.
- Contar historias parcialmente terminadas para calcular Velocity.
- Usar Spikes para todo en vez de reservarlos para incertidumbres relevantes.
- Cambiar la escala de Story Points una vez adoptada por el equipo.

Fuentes del material

Este apunte fue elaborado a partir exclusivamente de los dos materiales de clase proporcionados para esta actividad:

- “Requerimientos Ágiles / User Stories” — Ingeniería y Calidad de Software, UTN Facultad Regional Córdoba, Ciclo Lectivo 2026.
- “Estimaciones en Ambientes Ágiles” — Ingeniería y Calidad de Software, UTN Facultad Regional Córdoba, 2026.

Bibliografía de referencia indicada en los materiales: Mike Cohn, User Stories Applied; Mike Cohn, Agile Estimating and Planning; Steve McConnell, Software Estimation: Demystifying the Black Art; Bill Wake, INVEST in Stories.